



LABORATÓRIO PRÁTICO

Análise de Pacotes na Prática com Wireshark

Captura e Análise de Pacotes de Rede em Kali Linux

Prof. Dr. Laerte Peotta de Melo

Duração: 2 horas • Formato: prático, individual • Alvo: unb.br + serviço local

Agenda da Aula

PARTE TEÓRICA

- 01 Por que Capturar e Analisar Pacotes
- 02 Sniffing Ativo x Sniffing Passivo
- 03 Dois Alvos, Um Ambiente
- 04 Por que Capturar sem Root

PARTE PRÁTICA - LABORATÓRIO

- 05 Atividade 0 - Preparando o Ambiente
- 06 Módulo 1 - Fundamentos de Captura
- 07 Módulo 2 - Protocolos em HTTPS Real
- 08 Módulo 3 - Tráfego Local Simulado
- 09 Módulo 4 - Estudo de Caso Guiado
- 10 Módulo 5 - Negação de Serviço (DoS)

Roteiro completo: github.com/peotta/lab-wireshark-unb/roteiro-aluno.md



Para Estudar Além desta Aula



Capa da edição brasileira

Wireshark Docs wireshark.org/docs - documentação oficial completa

Sample Captures wiki.wireshark.org/SampleCaptures - pcaps reais para praticar

Referência bibliográfica

SANDERS, Chris. Análise de Pacotes na Prática: usando Wireshark para solucionar problemas de rede do mundo real. São Paulo: Novatec. Original: Practical Packet Analysis (No Starch Press).

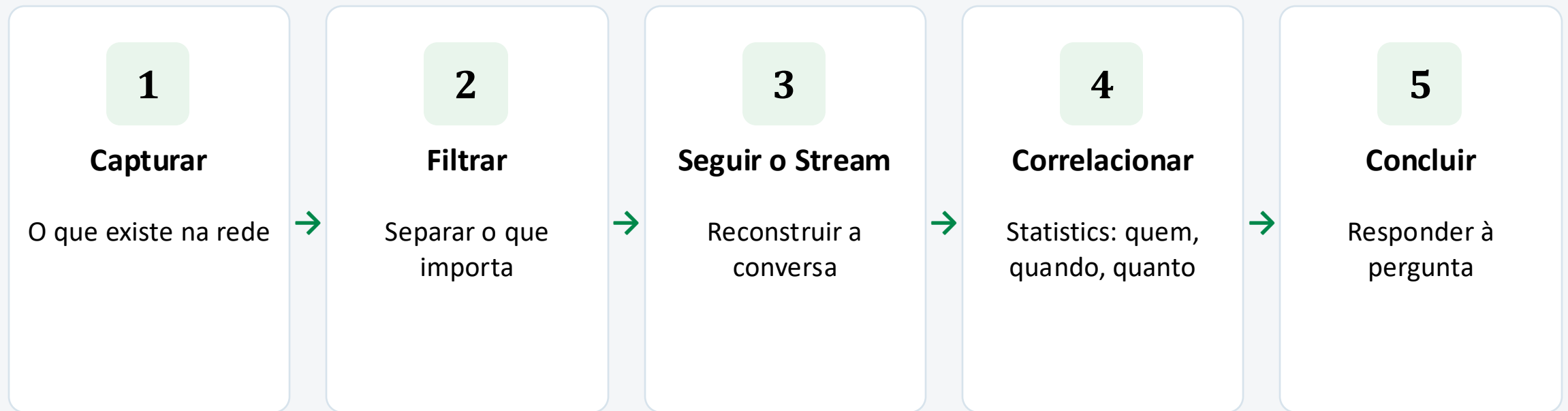
Repositório do Laboratório github.com/peotta/lab-wireshark-unb

Mãos ao Wireshark

Prof. Dr. Laerte Peotta de Melo

Por que Capturar e Analisar Pacotes?

Todo pacote que atravessa a rede conta uma história - o Wireshark é a lupa que deixa essa história legível.



Esse fluxo mental vai se repetir em cada atividade prática da aula.

Sniffing Ativo x Sniffing Passivo

A diferença não está na ferramenta - está em alterar ou não o que passa pela rede.



INTERCEPTAÇÃO SILENCIOSA

Sniffing Passivo

O quê: Escutar o tráfego que já passa pela interface, sem alterar nada na rede

Como: Interface em modo promíscuo, só leitura: Wireshark, tcpdump, ngrep

Neste curso: Módulos 1 a 4 - DNS, TCP, TLS e credenciais em texto claro

Rastro: Nenhum - é pura escuta



MANIPULAÇÃO DA REDE

Sniffing Ativo

O quê: Injetar, forjar ou manipular pacotes para forçar tráfego a aparecer

Como: ARP/DHCP spoofing, pacotes forjados: SYN flood, IP spoofing

Neste curso: Módulo 5 - SYN flood e spoofing de origem, contra a própria máquina

Rastro: Sempre deixa: volume anômalo, pacotes forjados

Neste curso, os Módulos 1 a 4 são 100% passivos. Só o Módulo 5 introduz uma técnica ativa - e só contra a própria máquina.

Dois Alvos, Um Ambiente

Tudo roda na máquina do aluno - sem Docker, sem servidor centralizado.



Tráfego Real - unb.br

Acessado via interface com internet (eth0/wlan0). Gera DNS, handshake TCP e TLS reais - mas nunca técnica ativa contra ele. Só navegação legítima.



Tráfego Simulado - localhost

Serviço que o próprio aluno sobe (FTP nativo ou login_server.py em Python), capturado na interface Loopback: lo. É onde acontecem credenciais em texto claro, força bruta e DoS.

Por que Capturar sem Root

Princípio do menor privilégio, aplicado ao próprio Wireshark.



Superfície de Ataque

Dissectors de protocolo processam pacotes não confiáveis. Um bug em um parser rodando como root vira execução de código como root.



Separação de Privilégio

Só o dumpcap precisa de CAP_NET_RAW/CAP_NET_ADMIN (grupo wireshark) - não a interface gráfica inteira.



Prática Recomendada

A própria documentação oficial do Wireshark recomenda não rodar como root, pelos dois motivos acima.

PLANO B, PONTUAL

Se algum aluno chegar sem o grupo configurado: "sudo wireshark" resolve na hora, só para aquele aluno - aceitável numa VM de laboratório descartável, não em produção.

ATIVIDADE 0

Preparando o Ambiente



```
$ wget .../scripts/setup-ambiente.sh  
$ chmod +x setup-ambiente.sh  
$ ./setup-ambiente.sh
```

```
# configura grupo wireshark,  
# instala vsftpd e hping3,  
# cria usuario aluno_teste,  
# baixa login_server.py
```



Repositório do Laboratório

github.com/peotta/lab-wireshark-unb - todo script pode ser baixado com wget, sem copiar/colar.

- ✓ Wireshark abre sem erro de permissão
- ✓ `curl -I https://unb.br` responde
- ✓ Serviço local sobe e responde em localhost
- ✓ Interface lo aparece na lista do Wireshark

Comandos Essenciais

```
ip a
```

SETUP

Identificar a interface de rede a capturar

```
dig unb.br
```

MÓD. 2

Forçar uma consulta DNS

```
curl -I https://unb.br
```

MÓD. 2

Gerar handshake TCP/TLS

```
sudo tcpdump -i <if> -w arq.pcap
```

MÓD. 1

Captura headless, sem GUI

```
python3  
login_server.py
```

MÓD. 3

Subir o serviço HTTP local

```
curl -d  
"user=X&pass=Y" ...
```

MÓD. 3/4

Simular um login no serviço local

```
sudo hping3 -S -p 8080  
--flood localhost
```

MÓD. 5

SYN flood contra o próprio serviço

```
sudo pkill hping3
```

MÓD. 5

Encerrar o flood

Wireshark na Prática: Demonstração do Professor



```
$ wireshark &  
$ ping -c 4 unb.br  
$ curl -I https://unb.br
```



Roteiro da Demonstração

- 1 Escolher a interface de rede e iniciar a captura
- 2 Percorrer os três painéis: lista, detalhes e bytes
- 3 Aplicar um filtro simples: icmp e depois tls
- 4 Usar Follow TCP Stream em uma conexão
- 5 Abrir Statistics: Protocol Hierarchy e Conversations

Primeira Captura Guiada

Objetivo: iniciar/parar captura e explorar os painéis do Wireshark.



```
$ ping -c 4 unb.br
```



Passo a Passo

- 1 Abrir Wireshark, selecionar interface com internet, Start
- 2 Rodar o ping em outro terminal
- 3 Parar a captura: ver ICMP Echo Request/Reply
- 4 Explorar: lista, detalhes, bytes brutos (hex)

CHECKPOINT

O IP público resolvido para unb.br deve estar visível na captura.

Filtro de Captura x Filtro de Exibição

Dois estágios diferentes: o que é gravado x o que é mostrado.



```
# Filtro de captura (BPF)
host unb.br
port 443
icmp

# Filtro de exibição
ip.addr == <ip-do-unb.br>
tcp.port == 443
dns
tls
```



Exercício

- 1 Navegar até <https://unb.br> capturando
- 2 Aplicar: `dns.qry.name contains "unb.br"`
- 3 Aplicar: `tcp.port == 443`
- 4 Aplicar: `tls.handshake.type == 1` (Client Hello)

Filtros e Operadores

`== / !=`

OPERADOR

Igual a / diferente de

`contains / matches`

OPERADOR

Contém um valor / bate com regex

`&& / || / !`

OPERADOR

E / ou / negação

`dns.qry.name contains "unb.br"`

MÓD. 2

Consultas DNS do domínio

`tls.handshake.type == 1`

MÓD. 2

Apenas o Client Hello

`http.request.method == "POST"`

MÓD. 3/4

Só envios de login

`tcp.flags.syn==1 && tcp.flags.ack==0`

MÓD. 5

SYN sem handshake completo

`tcp.analysis.retransmission`

GERAL

Retransmissões (perda/latência)

Salvando e Reabrindo Capturas

O formato .pcapng preserva tudo para análise posterior.



```
# Pela interface
File > Save As > arquivo.pcapng

# Alternativa headless
$ sudo tcpdump -i <interface> \
    host unb.br -w captura.pcap
```



Por que Salvar

Uma captura salva pode ser reaberta, compartilhada e reanalisada depois - inclusive fora da aula, ou usada como backup se o ambiente falhar durante o exercício.

tcpdump é útil quando não há interface gráfica disponível (servidores remotos, scripts automatizados).

DNS: o Primeiro Passo

Antes de qualquer conexão, o nome precisa virar endereço IP.



```
$ dig unb.br
```



No Wireshark

- 1 Filtro: dns
- 2 Observar a query e a response
- 3 Campos: dns.qry.name, dns.a
- 4 Se houver vários IPs: balanceamento de carga

TCP Three-Way Handshake

Toda conexão confiável começa com esses três pacotes.



```
$ curl -I https://unb.br
```



No Wireshark

- 1 Filtro: `ip.addr == <ip-unb.br> && tcp.port == 443`
- 2 Identificar SYN no campo `tcp.flags`
- 3 Identificar SYN, ACK (resposta do servidor)
- 4 Identificar ACK final (conexão estabelecida)

TLS Handshake: o que Fica Visível

HTTPS não é invisível - só protege parte do que trafega.



```
# Filtro
```

```
tls.handshake
```

```
# Client Hello
```

```
extensão SNI: unb.br em texto claro
```

```
# Server Hello / Certificate
```

```
emissor, validade, assinatura
```



Follow > TCP Stream

Num pacote de Application Data: o conteúdo é ilegível (cifrado).

PONTO CENTRAL

TLS protege o conteúdo, não os metadados. O domínio acessado ainda é visível para quem observa a rede.

Decifrando o Próprio HTTPS (opcional)

O navegador pode exportar as próprias chaves de sessão TLS.



```
$ export SSLKEYLOGFILE=~/.tls-keys.log  
$ firefox &
```

```
# No Wireshark:  
# Edit > Preferences > Protocols > TLS  
# (Pre)-Master-Secret log filename
```



Follow > HTTP Stream

Com o TLS keylog configurado, o conteúdo da página aparece decifrado dentro do próprio Wireshark.

POR QUE NÃO É QUEBRAR CRIPTOGRAFIA

É o próprio navegador exportando as próprias chaves - recurso oficial de depuração.

Subindo o Serviço Local

O alvo simulado, sem TLS por cima.



```
# Opção B: HTTP em Python puro
```

```
$ python3 login_server.py
```

```
# Opção A: FTP nativo
```

```
$ sudo systemctl start vsftpd
```



Escolha uma Opção

Ambas simulam um login sem TLS por cima - o mesmo tipo de requisição/resposta do Módulo 2, mas sem cifragem.

Nenhuma dependência extra além do Python 3, já incluído no Kali.

Capturando um Login Legítimo

O momento em que a diferença com HTTPS fica óbvia.



```
$ curl -d "user=admin&pass=Senha..." \  
http://localhost:8080/
```

ou, para FTP:

```
$ ftp localhost
```



No Wireshark

- 1 Capturar na interface Loopback: lo
- 2 Filtro: `http.request.method == "POST"`
- 3 Follow > TCP Stream
- 4 Usuário e senha aparecem em texto claro

Gerando o Incidente

Cenário: "Alguém tentou adivinhar a senha do serviço local."



```
$ for pass in 123456 senha admin \  
  SenhaSuperSecreta123; do  
  curl -s -d "user=admin&pass=$pass" \  
    http://localhost:8080/ > /dev/null  
done
```



Força Bruta, Contra Você Mesmo

O loop tenta várias senhas contra o próprio serviço local, gerando o tráfego que a captura vai precisar analisar na próxima atividade.

Investigando a Captura

Responder à pergunta de investigação com o que a captura mostra.



Passo a Passo

- 1 Filtro: `http.request.method == "POST"`
- 2 Follow > HTTP Stream em cada tentativa
- 3 Localizar a resposta "Login OK"

PERGUNTAS PARA RESPONDER

- Quantas tentativas houve e em qual intervalo de tempo?
- Qual foi a senha que funcionou?
- Que evidência no pcap comprova o sucesso do login?

Preparação e Regra de Ouro

Reconhecer a assinatura de tráfego de uma negação de serviço.

REGRA DE OURO - REPITA PARA SI MESMO

O alvo é sempre localhost/127.0.0.1, nunca outro IP. Nem unb.br, nem a máquina do colega, nem o gateway da rede do laboratório.



Antes de Começar

- ✓ Tirar um snapshot da VM (VirtualBox > Machine > Take Snapshot)
- ✓ Manter o login_server.py (ou vsftpd) do Módulo 3 rodando
- ✓ Confirmar hping3 disponível: `which hping3`

Linha de Base: Como é um SYN Normal

Antes de reconhecer uma anomalia, ver como é o normal.



```
$ curl http://localhost:8080/
```



No Wireshark

- 1 Capturar na interface Loopback: lo
- 2 Filtro: tcp.flags.syn == 1
- 3 Observar UM SYN e o handshake completo

Gerando o Flood

SYN flood contra o próprio serviço local.



```
$ sudo hping3 -S -p 8080 --flood \  
localhost
```

```
# Ctrl+C apos 10-15 segundos
```

```
# Em outro terminal, ao mesmo tempo:
```

```
$ curl -m 3 http://localhost:8080/
```

O QUE OBSERVAR

O curl de teste fica lento ou não responde dentro do timeout - o serviço está ocupado demais para atender pedidos legítimos.

Deixe rodando no máximo 10 a 15 segundos.

Analizando a Captura do Flood

Não é sobre o conteúdo do pacote - é sobre o padrão.



```
# Filtro
tcp.flags.syn == 1 && \
  tcp.flags.ack == 0

# Statistics
IO Graph → pico abrupto
Conversations → centenas half-open
```



Discussão Guiada

- Por que um volume alto de SYN sem ACK final esgota recursos do servidor?
- Por que isso é visualmente diferente do login legítimo do Módulo 3?
- Como um firewall/IDS usaria esse mesmo padrão para detectar e bloquear automaticamente?

Spoofing de Origem e Encerramento

Simulando múltiplas origens falsas, a partir de uma única máquina.



```
$ sudo hping3 -S -p 8080 \
    --rand-source --flood localhost
```

```
# Encerrando o ambiente
```

```
$ sudo pkill hping3
```



No Wireshark

Statistics > Endpoints mostra dezenas de IPs de origem distintos - parece um ataque distribuído, mesmo saindo de uma única máquina.

GANCHO PARA DEFESA REAL

A defesa contra spoofing não acontece no alvo - acontece na origem, via BCP38/egress filtering, bloqueando pacotes com IP de origem que não é da própria rede.

Três Padrões, Três Assinaturas

O mesmo Wireshark, revelando três tipos diferentes de tráfego.



MÓDULO 2

Tráfego Real

SNI visível, conteúdo cifrado - metadados expostos, dados protegidos.



MÓDULO 3

Serviço Local

Requisição/resposta em texto claro - credenciais visíveis linha a linha.



MÓDULO 5

Negação de Serviço

Volume anômalo de SYN sem ACK - o padrão importa mais que o conteúdo.

O Wireshark não julga se o tráfego é malicioso - ele te dá os fatos. A interpretação é sua.



FIM DA AULA

Obrigado!

Obrigado pela participação - agora é praticar: capture, filtre e interprete.

Prof. Dr. Laerte Peotta de Melo

Material e roteiro: github.com/peotta/lab-wireshark-unb